Kubernetes YAML

Complete Line-by-Line Guide

`release/kubernetes-manifests.yaml`

991 lines | 12 services | Every line explained

DevOps Shack | @devopsshack

How to Read This Document

Each section of the YAML is shown as a code block (dark background, monospace font).

Below each code block, every line is explained in plain English — what the field does and why it matters.

Sections are grouped by service. Each service has 3 objects: Deployment, Service, and ServiceAccount.

```
# Yellow text = YAML comments from the original file
apiVersion: apps/v1   # blue label = the YAML line being explained below
```

The explanations directly below each block are labeled with the field name so you can match them easily.

1. Currency Service

Converts money amounts between supported currencies using live rates from the European Central Bank. Written in Node.js. Highest requests-per-second service in the entire system. Internal only — port 7000.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: currencyservice
  labels:
    app: currencyservice
spec:
  selector:
    matchLabels:
      app: currencyservice
  template:
    metadata:
      labels:
        app: currencyservice
    spec:
      serviceAccountName: currencyservice
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop:
              - ALL
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/currencyservice:887ad4c5...
        ports:
        - name: grpc
          containerPort: 7000
        env:
        - name: PORT
          value: "7000"
```

```
        - name: DISABLE_PROFILER
          value: "1"
        readinessProbe:
          grpc:
            port: 7000
        livenessProbe:
          grpc:
            port: 7000
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
```

Line-by-Line Explanation

`---` YAML document separator. Divides multiple Kubernetes objects in one file. Each --- starts a new object.

`apiVersion: apps/v1` Which Kubernetes API handles this object. apps/v1 is the stable, production version for Deployments.

`kind: Deployment` Object type. A Deployment manages a group of identical pods — it handles restarts, rolling updates, and scaling.

`metadata.name: currencyservice` The unique name of this Deployment. Used in kubectl commands: kubectl get deployment currencyservice.

`metadata.labels: app: currencyservice` Tags on the object. The Service uses this label to find which pods to route traffic to.

`spec.selector.matchLabels` Tells the Deployment which pods it owns and should manage. Must match the pod template labels exactly.

`spec.template.metadata.labels` Labels stamped on every pod this Deployment creates. Must match selector — Kubernetes rejects it if they don't match.

`serviceAccountName: currencyservice` Assigns the currencyservice Kubernetes identity to this pod. Controls what the pod can access in the Kubernetes API.

`imagePullSecrets: ghcr-secret` The secret containing credentials for GitHub Container Registry (ghcr.io). Without it, the image pull fails with 401 Unauthorized.

`terminationGracePeriodSeconds: 5` When shutting down, Kubernetes waits 5 seconds for in-flight requests to complete before force-killing the process.

`securityContext.fsGroup: 1000` Files in mounted volumes are owned by group 1000, so the container process can read/write them without root access.

`securityContext.runAsGroup: 1000` The container process runs as Linux group ID 1000 — not root (group 0).

`securityContext.runAsNonRoot: true` Kubernetes REJECTS the pod if the container tries to run as root (UID 0). Hard security guardrail — cannot be bypassed by the image.

`securityContext.runAsUser: 1000` Container runs as user ID 1000, a regular unprivileged user. Cannot access root-only system resources.

`securityContext.allowPrivilegeEscalation: false` The process can NEVER gain more Linux privileges than it started with. Prevents sudo/setuid attacks even if the binary is exploited.

`capabilities.drop: ALL` Removes ALL Linux kernel capabilities from the container (network admin, raw sockets, system tracing, etc.). The container gets zero OS-level powers — maximum isolation.

`privileged: false` Container does NOT run in privileged mode. A privileged container has full access to the host's kernel and devices — extremely dangerous. This ensures it cannot.

`readOnlyRootFilesystem: true` Container filesystem is read-only. Attackers who gain code execution cannot write malware or modify files at runtime. Temp writes must use mounted volumes.

`image: ghcr.io/...currencyservice:887ad4c5` The Docker image to run. The hash (887ad4c5...) is a git commit SHA — guarantees the exact same image on every deploy, fully reproducible.

`containerPort: 7000` The gRPC server inside the container listens on port 7000. Other pods call this service on this port.

`env PORT: "7000"` Injects PORT=7000 into the container. The app reads this environment variable at startup to know which port to bind on.

`env DISABLE_PROFILER: "1"` Turns off the Google Cloud Profiler agent. Set to 1 (disabled) because this is a self-hosted deploy — the profiler only makes sense on Google Cloud.

`readinessProbe.grpc.port: 7000` Kubernetes calls the gRPC health check on port 7000 before sending any traffic. If it fails, this pod is removed from the load balancer until healthy.

`livenessProbe.grpc.port: 7000` Kubernetes calls this periodically while the pod runs. If it fails repeatedly, Kubernetes RESTARTS the pod automatically — the self-healing mechanism.

`resources.requests.cpu: 100m` Requests 100 millicores of CPU (1000m = 1 full core). The Kubernetes scheduler uses this to find a node with enough free CPU before placing the pod.

`resources.requests.memory: 64Mi` Requests 64 MiB of RAM. The scheduler ensures this is available on the chosen node.

`resources.limits.cpu: 200m` Hard ceiling of 200 millicores. If the process uses more, Linux throttles it — it slows down but does not crash.

`resources.limits.memory: 128Mi` Hard ceiling of 128 MiB. If the process exceeds this, Kubernetes immediately kills and restarts it (OOMKill).

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: currencyservice
  labels:
    app: currencyservice
spec:
  type: ClusterIP
  selector:
    app: currencyservice
  ports:
  - name: grpc
    port: 7000
    targetPort: 7000
```

Line-by-Line Explanation

`apiVersion: v1` v1 is the core Kubernetes API. Services, ConfigMaps, Secrets, and ServiceAccounts all use v1.

`kind: Service` A Service gives pods a stable DNS name and IP. Even when pods restart and get new IPs, the Service address stays the same.

`metadata.name: currencyservice` The DNS name inside the cluster. Other pods call this service using currencyservice:7000. Kubernetes DNS resolves 'currencyservice' automatically.

`type: ClusterIP` Internal-only. Gets a virtual IP reachable only from inside the cluster. NOT accessible from the internet. Most secure type — the default.

`selector: app: currencyservice` Routes traffic to pods with this label. This is what connects the Service to the Deployment's pods.

`port: 7000` The port callers use. Frontend calls currencyservice:7000 — this 7000 is what it hits.

`targetPort: 7000` The port on the actual pod that receives the traffic. Kubernetes maps: caller → Service port 7000 → pod port 7000. They match here, but can differ (see emailservice).

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: currencyservice
```

A dedicated Kubernetes identity for currencyservice pods. Pods use their ServiceAccount token to authenticate to the Kubernetes API. Giving each service its own account enables fine-grained RBAC — you can grant permissions to one service without affecting others.

2. Load Generator

Simulates real user traffic (browse, add to cart, checkout) against the frontend using Python Locust. This is NOT a business microservice — it is a testing/load tool. It has no Service object because nothing calls it — it only makes outgoing requests.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: loadgenerator
  labels:
    app: loadgenerator
spec:
  selector:
    matchLabels:
      app: loadgenerator
  replicas: 1
  template:
    metadata:
      labels:
        app: loadgenerator
      annotations:
        sidecar.istio.io/rewriteAppHTTPProbers: "true"
    spec:
      serviceAccountName: loadgenerator
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      restartPolicy: Always
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      initContainers:
      - name: frontend-check
        image: busybox:latest
        command: ['/bin/sh', '-exc', '...script...']
        # Script logic:
        #   MAX_RETRIES=12
        #   RETRY_INTERVAL=10
        #   for i in 1..12:
        #     wget http://frontend:80 → get HTTP status code
        #     if status == 200 → exit 0 (success, start main container)
        #     else → sleep 10, retry
        #   after 12 failures → exit 1 (pod stays in init loop)
```

```
        env:
        - name: FRONTEND_ADDR
          value: "frontend:80"
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
      containers:
      - name: main
        image: ghcr.io/jaiswaladi246/loadgenerator:887ad4c5...
        env:
        - name: FRONTEND_ADDR
          value: "frontend:80"
        - name: USERS
          value: "10"
        - name: RATE
          value: "1"
        resources:
          requests:
            cpu: 300m
            memory: 256Mi
          limits:
            cpu: 500m
            memory: 512Mi
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
```

Line-by-Line Explanation

`replicas: 1` Run only 1 copy of the load generator. Unlike real services that need multiple replicas for availability, one load generator is enough to drive demo-scale traffic.

`annotations: sidecar.istio.io/rewriteAppHTTPProbers` An Istio service mesh annotation. If Istio is installed, it injects a proxy sidecar into every pod. This annotation tells Istio to rewrite HTTP health probes to work correctly through the proxy. Has no effect if Istio is not installed.

`restartPolicy: Always` If the pod exits for any reason, always restart it. This is the default for Deployments — ensures the load generator keeps running continuously.

`initContainers` A special container that runs BEFORE the main container starts. The main container does NOT start until the init container exits with code 0 (success).

`name: frontend-check` Name of the init container. Its job is to wait until the frontend is responding before starting the load generator. Prevents traffic from hitting a not-yet-ready store.

`image: busybox:latest` A tiny (~1 MB) Linux image with basic shell tools — wget, sleep, echo. Used only for the health check script, not the actual load generator.

`Script: MAX_RETRIES=12, RETRY_INTERVAL=10` Tries up to 12 times, waiting 10 seconds between each attempt. Maximum wait = 120 seconds (2 minutes) before giving up.

`Script: wget http://frontend:80` Makes an HTTP request to the frontend service. 'frontend:80' is Kubernetes DNS — resolves to the frontend ClusterIP automatically.

`Script: if status == 200 → exit 0` If the frontend returned HTTP 200 OK, the init container succeeds (exit 0), and Kubernetes immediately starts the main load generator container.

`Script: exit 1 after 12 failures` If frontend never responds after 12 attempts, exit with error (exit 1). Kubernetes marks the init container as failed and retries the whole pod.

`env FRONTEND_ADDR: frontend:80` The address the load generator sends all simulated traffic to. Kubernetes DNS resolves 'frontend' to the frontend Service's ClusterIP.

`env USERS: 10` Tells Locust to simulate 10 concurrent users browsing simultaneously. Increase this number to generate more load.

`env RATE: 1` Spawn 1 new simulated user per second until the USERS limit is reached. After 10 seconds, all 10 users are active at once.

`requests.cpu: 300m / memory: 256Mi` Load generator needs 3x-4x more resources than most services because Locust processes HTTP responses for every simulated user concurrently.

`limits.cpu: 500m / memory: 512Mi` Higher limits than all other services. 512Mi for memory because Locust tracks connection state and response timing for each simulated user session.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: loadgenerator
```

Dedicated Kubernetes identity for the load generator. Note: there is NO Service object for loadgenerator — it is a client that only makes outgoing requests. Nothing in the cluster calls it, so no Service is needed.

3. Product Catalog Service

Serves product data (list, search, get by ID) from a static JSON file baked into the Docker image. No database required — fully stateless. Written in Go. Port 3550.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: productcatalogservice
  labels:
    app: productcatalogservice
spec:
  selector:
    matchLabels:
      app: productcatalogservice
  template:
    metadata:
      labels:
        app: productcatalogservice
    spec:
      serviceAccountName: productcatalogservice
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/productcatalogservice:887ad4c5...
        ports:
        - containerPort: 3550
        env:
        - name: PORT
          value: "3550"
        - name: DISABLE_PROFILER
          value: "1"
        readinessProbe:
```

```
          grpc:
            port: 3550
        livenessProbe:
          grpc:
            port: 3550
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
```

Line-by-Line Explanation

`containerPort: 3550` gRPC server port 3550. Different from other services to avoid port conflicts when running multiple services on the same node during development.

`No REDIS_ADDR env var` This service has no database connection. All product data (names, prices, descriptions, images) is stored in a products.json file baked inside the Docker image at build time.

`DISABLE_PROFILER: 1` Disables Google Cloud Profiler. Not needed outside GCP environment.

`readinessProbe.grpc.port: 3550` gRPC health check on port 3550. Kubernetes removes this pod from the load balancer if it fails.

`livenessProbe.grpc.port: 3550` If this fails repeatedly, Kubernetes restarts the pod.

`limits.memory: 128Mi` Very low memory limit — the product JSON is small and Go is extremely memory-efficient. The entire running service uses under 50MB typically.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: productcatalogservice
  labels:
    app: productcatalogservice
spec:
  type: ClusterIP
  selector:
    app: productcatalogservice
  ports:
  - name: grpc
    port: 3550
    targetPort: 3550
```

Line-by-Line Explanation

`name: productcatalogservice` DNS name used by other services: productcatalogservice:3550. Both frontend and recommendationservice use this address.

`type: ClusterIP` Internal only — not accessible from outside the cluster.

`port: 3550 / targetPort: 3550` Direct 1:1 mapping. Traffic on port 3550 forwarded to container port 3550.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: productcatalogservice
```

Dedicated Kubernetes identity for product catalog pods. Isolated from other services — cannot access Kubernetes resources granted to other ServiceAccounts.

4. Checkout Service

The most connected service — orchestrates the full purchase flow by calling 6 other services in sequence. When a user places an order, checkout: fetches cart → gets prices → converts currency → charges payment → arranges shipping → sends confirmation email → empties cart. Written in Go. Port 5050.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: checkoutservice
  labels:
    app: checkoutservice
spec:
  selector:
    matchLabels:
      app: checkoutservice
  template:
    metadata:
      labels:
        app: checkoutservice
    spec:
      serviceAccountName: checkoutservice
      imagePullSecrets:
        - name: ghcr-secret
      # No terminationGracePeriodSeconds — uses K8s default of 30s
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
        - name: server
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
            privileged: false
            readOnlyRootFilesystem: true
          image: ghcr.io/jaiswaladi246/checkoutservice:887ad4c5...
          ports:
          - containerPort: 5050
          readinessProbe:
            grpc:
              port: 5050
          livenessProbe:
```

```
          grpc:
            port: 5050
        env:
        - name: PORT
          value: "5050"
        - name: PRODUCT_CATALOG_SERVICE_ADDR
          value: "productcatalogservice:3550"
        - name: SHIPPING_SERVICE_ADDR
          value: "shippingservice:50051"
        - name: PAYMENT_SERVICE_ADDR
          value: "paymentservice:50051"
        - name: EMAIL_SERVICE_ADDR
          value: "emailservice:5000"
        - name: CURRENCY_SERVICE_ADDR
          value: "currencyservice:7070"
        - name: CART_SERVICE_ADDR
          value: "cartservice:7070"
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
```

Line-by-Line Explanation

`No terminationGracePeriodSeconds` Unlike other services (which set 5s), checkout uses the Kubernetes default of 30 seconds. A checkout may be mid-payment when shutdown is requested — the extra time lets it finish safely.

`containerPort: 5050` Checkout's gRPC server listens on port 5050. Frontend calls checkoutservice:5050 when the user clicks 'Place Order'.

`PRODUCT_CATALOG_SERVICE_ADDR: productcatalogservice:3550` Checkout calls product catalog to get current prices for each item in the cart during order calculation.

`SHIPPING_SERVICE_ADDR: shippingservice:50051` Checkout calls shipping twice: once for GetQuote (calculate cost) and once for ShipOrder (finalize delivery).

`PAYMENT_SERVICE_ADDR: paymentservice:50051` Checkout calls payment to charge the credit card. This is the critical step — if payment fails, the order is aborted.

`EMAIL_SERVICE_ADDR: emailservice:5000` PORT 5000 — not 8080! The emailservice Service exposes port 5000 and internally maps it to container port 8080. Checkout always calls the Service port, not the container port.

`CURRENCY_SERVICE_ADDR: currencyservice:7070` Checkout calls currency to convert all prices to the user's selected currency before calculating the final order total.

`CART_SERVICE_ADDR: cartservice:7070` Checkout calls cart twice: GetCart (to retrieve items) and EmptyCart (to clear the cart after payment succeeds).

`6 downstream addresses` Every address is injected as an env var — never hardcoded. To change where any service lives, you edit the YAML and redeploy. No code change needed.

`resources: cpu 100m / memory 64Mi` Standard Go service budget. Checkout is mostly doing network I/O (calling other services) — it doesn't need high CPU or RAM itself.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: checkoutservice
  labels:
    app: checkoutservice
spec:
  type: ClusterIP
  selector:
    app: checkoutservice
  ports:
  - name: grpc
    port: 5050
    targetPort: 5050
```

Line-by-Line Explanation

`name: checkoutservice` DNS name. Frontend calls checkoutservice:5050 when the user submits the order form.

`type: ClusterIP` Internal only — checkout is never called directly from the internet.

`port: 5050 / targetPort: 5050` Direct mapping. Traffic arrives on 5050 and is forwarded to container port 5050.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: checkoutservice
```

Isolated Kubernetes identity for checkout pods. Because checkout calls 6 services, giving it its own account means you can audit exactly what checkout is doing in the cluster.

5. Shipping Service

Calculates shipping cost estimates and simulates order fulfillment (returns a fake tracking ID). A leaf service — it does not call any other service. Written in Go. Port 50051.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shippingservice
  labels:
    app: shippingservice
spec:
  selector:
    matchLabels:
      app: shippingservice
  template:
    metadata:
      labels:
        app: shippingservice
    spec:
      serviceAccountName: shippingservice
      imagePullSecrets:
        - name: ghcr-secret
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/shippingservice:887ad4c5...
        ports:
        - containerPort: 50051
        env:
        - name: PORT
          value: "50051"
        - name: DISABLE_PROFILER
          value: "1"
        readinessProbe:
          periodSeconds: 5
```

```
          grpc:
            port: 50051
        livenessProbe:
          grpc:
            port: 50051
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
```

Line-by-Line Explanation

`containerPort: 50051` Standard gRPC port. Same port as paymentservice — fine because each service runs in its own isolated pod with separate networking.

`readinessProbe.periodSeconds: 5` Check readiness every 5 seconds — more frequent than the default 10 seconds. Shipping needs to come online quickly after startup.

`livenessProbe (no periodSeconds)` Uses the default interval of 10 seconds for liveness. Only readiness uses the faster 5-second interval.

`No downstream env vars` Shipping is a leaf service — it receives requests and responds, but calls nothing else. This makes it simpler and independent.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: shippingservice
  labels:
    app: shippingservice
spec:
  type: ClusterIP
  selector:
    app: shippingservice
  ports:
  - name: grpc
    port: 50051
    targetPort: 50051
```

Line-by-Line Explanation

`name: shippingservice` DNS name. Both frontend (for quotes) and checkout (for actual shipping) call shippingservice:50051.

`port / targetPort: 50051` Direct 1:1 mapping to container.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: shippingservice
```

Isolated Kubernetes identity for shipping pods.

6. Cart Service

The only stateful microservice in the system. Stores shopping cart data (user ID → items) in Redis. Written in C#/.NET. Port 7070. Has a longer initialDelaySeconds on health probes because C# and Redis connection setup both take time.

Deployment

YAML

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cartservice
  labels:
    app: cartservice
spec:
  selector:
    matchLabels:
      app: cartservice
  template:
    metadata:
      labels:
        app: cartservice
    spec:
      serviceAccountName: cartservice
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/cartservice:887ad4c5...
        ports:
        - containerPort: 7070
        env:
        - name: REDIS_ADDR
          value: "redis-cart:6379"
        resources:
          requests:
```

```
            cpu: 200m
            memory: 64Mi
          limits:
            cpu: 300m
            memory: 128Mi
        readinessProbe:
          initialDelaySeconds: 15
          grpc:
            port: 7070
        livenessProbe:
          initialDelaySeconds: 15
          periodSeconds: 10
          grpc:
            port: 7070
```

Line-by-Line Explanation

`containerPort: 7070` gRPC port 7070. Different from most services to avoid conflict with other gRPC services in development environments.

`env REDIS_ADDR: redis-cart:6379` The ONLY env var for cart. 'redis-cart' is the Kubernetes DNS name of the Redis Service. Port 6379 is the standard Redis port. Cart serializes cart data as protobuf bytes and stores it with the user's session ID as the Redis key.

`requests.cpu: 200m` Higher than most services (200m vs 100m). C#/.NET runtime has more CPU overhead than Go.

`requests.memory: 64Mi / limits: 128Mi` Cart itself is memory-efficient — Redis holds the actual data. The C# service only holds in-flight request state.

`readinessProbe.initialDelaySeconds: 15` CRITICAL: Wait 15 seconds before first health check. The C# .NET runtime takes time to initialize, AND it needs to establish a connection to Redis before it can serve requests. Without this delay, Kubernetes probes too early, sees failure, and restarts the pod in a loop.

`livenessProbe.initialDelaySeconds: 15` Same 15-second wait for liveness. If cart later loses its Redis connection and stays down, Kubernetes will restart it after 3 × 10 = 30 seconds.

`livenessProbe.periodSeconds: 10` Check liveness every 10 seconds. Explicit here (matches default) for clarity.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: cartservice
  labels:
    app: cartservice
spec:
  type: ClusterIP
  selector:
    app: cartservice
```

```
  ports:
  - name: grpc
    port: 7070
    targetPort: 7070
```

`name: cartservice` DNS name. Frontend calls cartservice:7070 to read/write the cart. Checkout also calls cartservice:7070 to fetch and then empty the cart.

`port / targetPort: 7070` Direct 1:1 port mapping.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cartservice
```

Dedicated identity for cart pods. Because cart is stateful and handles user session data, isolating its identity is important for security auditing.

7. Redis (redis-cart)

The database backend for cartservice. An in-memory key-value store. Uses the official Redis Alpine image — no custom code. Port 6379. Unlike the microservices, Redis has no ServiceAccount, and uses a tcpSocket health probe (not gRPC) and a volume for data persistence.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-cart
  labels:
    app: redis-cart
spec:
  selector:
    matchLabels:
      app: redis-cart
  template:
    metadata:
      labels:
        app: redis-cart
    spec:
      # No serviceAccountName — Redis doesn't need a K8s identity
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: redis
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: redis:alpine
        ports:
        - containerPort: 6379
        readinessProbe:
          periodSeconds: 5
          tcpSocket:
            port: 6379
        livenessProbe:
          periodSeconds: 5
          tcpSocket:
            port: 6379
```

```
        volumeMounts:
        - mountPath: /data
          name: redis-data
        resources:
          limits:
            memory: 256Mi
            cpu: 125m
          requests:
            cpu: 70m
            memory: 200Mi
      volumes:
      - name: redis-data
        emptyDir: {}
```

Line-by-Line Explanation

`No serviceAccountName` Redis doesn't need a Kubernetes identity because it never talks to the Kubernetes API. Omitting it is intentional — minimal permissions.

`image: redis:alpine` The official Redis image built on Alpine Linux. Alpine is tiny (~7MB vs ~120MB for Debian). No customization needed — Redis runs as-is.

`containerPort: 6379` Standard Redis port. The cartservice connects here using redis-cart:6379.

`tcpSocket probe (not grpc)` Redis doesn't speak the gRPC health check protocol. A tcpSocket probe simply checks if the port is accepting TCP connections — sufficient for Redis.

`readinessProbe.periodSeconds: 5` Check every 5 seconds. Redis starts quickly so frequent checks get it into service fast.

`livenessProbe.periodSeconds: 5` Also every 5 seconds for liveness. If Redis stops accepting connections, the pod is restarted within ~15 seconds.

`volumeMounts.mountPath: /data` Redis writes its data files (RDB snapshots) to /data. Without this mount, data would go to the read-only root filesystem and Redis would crash.

`volumes.emptyDir: {}` An empty directory created on the node's disk when the pod starts. DATA SURVIVES container restarts within the same pod, but is PERMANENTLY LOST if the pod is deleted, crashes, or moves to a different node. Acceptable for a demo — use a PersistentVolume in production.

`resources.limits first` In this block, limits are listed before requests — YAML field order doesn't matter to Kubernetes. Both sections are processed the same way.

`requests.cpu: 70m / limits: 125m` Redis is mostly idle between cart operations — very low CPU. Scheduled with 70m guaranteed, capped at 125m.

`requests.memory: 200Mi / limits: 256Mi` Redis pre-allocates memory. Request of 200Mi is close to the 256Mi limit — Redis needs most of its ceiling just for the runtime.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: redis-cart
```

```
  labels:
    app: redis-cart
spec:
  type: ClusterIP
  selector:
    app: redis-cart
  ports:
  - name: tcp-redis
    port: 6379
    targetPort: 6379
```

`name: redis-cart` The DNS name that cartservice uses in its REDIS_ADDR env var: redis-cart:6379. The name makes clear this is cart's dedicated Redis — not a shared Redis.

`name: tcp-redis` Port named 'tcp-redis' (not 'grpc') — accurately describes the plain TCP protocol Redis uses.

8. Email Service

Sends mock order confirmation emails (logs to stdout, no real SMTP). Python with async gRPC. Has the most detailed probe configuration of all services. IMPORTANT: The container listens on port 8080, but the Service exposes port 5000 — Kubernetes translates 5000 → 8080 internally.

Deployment

YAML

```yaml
---
---
# Two --- separators in the original file — harmless duplicate
apiVersion: apps/v1
kind: Deployment
metadata:
  name: emailservice
  labels:
    app: emailservice
spec:
  selector:
    matchLabels:
      app: emailservice
  template:
    metadata:
      labels:
        app: emailservice
    spec:
      serviceAccountName: emailservice
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/emailservice:2982d340...
        ports:
        - containerPort: 8080
        env:
        - name: PORT
          value: "8080"
```

```
        - name: DISABLE_PROFILER
          value: "1"
        readinessProbe:
          grpc:
            port: 8080
          initialDelaySeconds: 10
          periodSeconds: 5
          timeoutSeconds: 3
          failureThreshold: 3
        livenessProbe:
          grpc:
            port: 8080
          initialDelaySeconds: 15
          periodSeconds: 10
          timeoutSeconds: 5
          failureThreshold: 3
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
```

Line-by-Line Explanation

`image SHA: 2982d340...` DIFFERENT commit SHA from most services (887ad4c5...). Email service was updated independently and deployed from its own separate git commit.

`containerPort: 8080` The gRPC server inside the container binds to port 8080. BUT — callers use port 5000 (the Service port, see below). Kubernetes translates.

`env PORT: 8080` App binds internally to 8080.

`readinessProbe.initialDelaySeconds: 10` Wait 10 seconds before first readiness check. Python gRPC async servers (using grpc.experimental.aio) take longer to initialize than Go services.

`readinessProbe.periodSeconds: 5` Check readiness every 5 seconds — fast detection if the service isn't ready yet.

`readinessProbe.timeoutSeconds: 3` If the health check doesn't respond within 3 seconds, count it as failed. Short timeout = fast failure detection.

`readinessProbe.failureThreshold: 3` Remove from load balancer after 3 consecutive failures. Time to action = 3 × 5s = 15 seconds of failures.

`livenessProbe.initialDelaySeconds: 15` Wait 15 seconds before first liveness check — longer than readiness (10s). Give the service more time to fully start before risking a restart loop.

`livenessProbe.periodSeconds: 10` Check liveness every 10 seconds — less frequently than readiness. Reduces noise.

`livenessProbe.timeoutSeconds: 5` More lenient 5-second timeout for liveness vs 3 seconds for readiness.

`livenessProbe.failureThreshold: 3` Restart the pod after 3 consecutive liveness failures. Time to restart = 3 × 10s = 30 seconds.

Service — Port Translation: 5000 → 8080

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: emailservice
  labels:
    app: emailservice
spec:
  type: ClusterIP
  selector:
    app: emailservice
  ports:
  - name: grpc
    port: 5000       # <-- callers use this
    targetPort: 8080  # <-- container actually listens here
```

Line-by-Line Explanation

`name: emailservice` DNS name used by checkout: emailservice:5000. Note: port 5000, not 8080.

`port: 5000` The port CALLERS use. Checkout's EMAIL_SERVICE_ADDR is 'emailservice:5000'. This is a deliberate design choice — the public port is decoupled from the implementation.

`targetPort: 8080` Kubernetes TRANSLATES traffic arriving at port 5000 → forwards to container port 8080. This means the container can change its internal port without breaking callers — only the Service YAML needs updating.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: emailservice
```

Isolated Kubernetes identity for email service pods.

9. Payment Service

Simulates credit card payment processing. Validates card format using the Luhn algorithm, checks expiry, and returns a fake UUID transaction ID. Written in Node.js. Port 50051. Leaf service — calls nothing else. Has its own unique image SHA.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: paymentservice
  labels:
    app: paymentservice
spec:
  selector:
    matchLabels:
      app: paymentservice
  template:
    metadata:
      labels:
        app: paymentservice
    spec:
      serviceAccountName: paymentservice
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/paymentservice:e8fdd88e...
        ports:
        - containerPort: 50051
        env:
        - name: PORT
          value: "50051"
        - name: DISABLE_PROFILER
          value: "1"
```

```
        readinessProbe:
          grpc:
            port: 50051
        livenessProbe:
          grpc:
            port: 50051
        resources:
          requests:
            cpu: 100m
            memory: 64Mi
          limits:
            cpu: 200m
            memory: 128Mi
```

Line-by-Line Explanation

`image SHA: e8fdd88e...` DIFFERENT commit SHA — payment service was deployed from its own independent git commit, separately versioned from most other services.

`containerPort: 50051` Standard gRPC port. Same as shippingservice — fine because they run in separate isolated pods.

`No downstream env vars` Payment is a leaf service. It only receives requests (from checkout) and responds. It calls no other service.

`limits.memory: 128Mi` Node.js runtime needs slightly more memory than Go, but 128Mi is sufficient for this mock service.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: paymentservice
  labels:
    app: paymentservice
spec:
  type: ClusterIP
  selector:
    app: paymentservice
  ports:
  - name: grpc
    port: 50051
    targetPort: 50051
```

Line-by-Line Explanation

`name: paymentservice` DNS name. Checkout calls paymentservice:50051 to charge the credit card.

`type: ClusterIP` Internal only — payment is never exposed to the internet directly.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: paymentservice
```

Isolated Kubernetes identity for payment pods. Given the sensitivity of payment processing, keeping the identity isolated is important even in a mock service.

10. Frontend

The only public-facing service. An HTTP server (Go net/http) that renders HTML pages server-side and calls all 8 backend services. Container port 8080, Service port 80. Unique image SHA. Has TWO Kubernetes Services: one ClusterIP (internal) and one LoadBalancer (external, gets a public internet IP).

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
      annotations:
        sidecar.istio.io/rewriteAppHTTPProbers: "true"
    spec:
      serviceAccountName: frontend
      imagePullSecrets:
        - name: ghcr-secret
      # No terminationGracePeriodSeconds — uses 30s default
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
        - name: server
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
            privileged: false
            readOnlyRootFilesystem: true
          image: ghcr.io/jaiswaladi246/frontend:6ddc9ef0...
          ports:
          - containerPort: 8080
          readinessProbe:
            initialDelaySeconds: 10
            httpGet:
```

```
            path: "/_healthz"
            port: 8080
            httpHeaders:
            - name: "Cookie"
              value: "shop_session-id=x-readiness-probe"
          livenessProbe:
            initialDelaySeconds: 10
            httpGet:
              path: "/_healthz"
              port: 8080
              httpHeaders:
              - name: "Cookie"
                value: "shop_session-id=x-liveness-probe"
          env:
          - name: PORT
            value: "8080"
          - name: PRODUCT_CATALOG_SERVICE_ADDR
            value: "productcatalogservice:3550"
          - name: CURRENCY_SERVICE_ADDR
            value: "currencyservice:7000"
          - name: CART_SERVICE_ADDR
            value: "cartservice:7070"
          - name: RECOMMENDATION_SERVICE_ADDR
            value: "recommendationservice:8080"
          - name: SHIPPING_SERVICE_ADDR
            value: "shippingservice:50051"
          - name: CHECKOUT_SERVICE_ADDR
            value: "checkoutservice:5050"
          - name: AD_SERVICE_ADDR
            value: "adservice:9555"
          - name: SHOPPING_ASSISTANT_SERVICE_ADDR
            value: "shoppingassistantservice:80"
          # - name: ENV_PLATFORM
          #   value: "aws"   (commented out — optional)
          - name: ENABLE_PROFILER
            value: "0"
          # - name: CYMBAL_BRANDING     (commented out — optional)
          # - name: ENABLE_ASSISTANT    (commented out — optional)
          # - name: FRONTEND_MESSAGE    (commented out — optional)
          # - name: PACKAGING_SERVICE_URL (commented out — optional)
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 200m
              memory: 128Mi
```

Line-by-Line Explanation

`image SHA: 6ddc9ef0...` Unique commit SHA — frontend was deployed from its own separate git commit, independently from most services.

`annotation: sidecar.istio.io/rewriteAppHTTPProbers: true` Istio rewrites HTTP health probes to go through its sidecar proxy. Required for HTTP-based probes to work correctly when Istio is installed.

No terminationGracePeriodSeconds (uses 30s default) Frontend may be mid-page-render when shutdown happens. 30 seconds is enough to finish serving open requests.

containerPort: 8080 HTTP server binds to 8080 (non-privileged port — no root needed). The Service maps the standard HTTP port 80 to this 8080.

readinessProbe: httpGet (not gRPC) Frontend is an HTTP server, not gRPC. Uses an HTTP GET health check instead. All other services use the gRPC health protocol.

readinessProbe.path: /_healthz A dedicated health check URL that returns HTTP 200 OK when the server is ready. Not a real page — purely for Kubernetes probes.

Cookie: shop_session-id=x-readiness-probe The frontend requires a session cookie for every request. Without it, the app might reject the health check. This fake session value lets the health check pass without creating a real user session.

livenessProbe Cookie: x-liveness-probe Slightly different cookie value for liveness vs readiness. Allows distinguishing which probe type triggered a request in the application logs.

PRODUCT_CATALOG_SERVICE_ADDR: productcatalogservice:3550 Used to list and display products on the home page and product pages.

CURRENCY_SERVICE_ADDR: currencyservice:7000 Every product price displayed requires a currency conversion call. Frontend calls this for every page that shows prices.

CART_SERVICE_ADDR: cartservice:7070 Read and write cart for the cart page and during checkout flow.

RECOMMENDATION_SERVICE_ADDR: recommendationservice:8080 Fetches 'You may also like' products shown on product detail pages.

SHIPPING_SERVICE_ADDR: shippingservice:50051 Gets shipping cost estimate shown on the cart page before checkout.

CHECKOUT_SERVICE_ADDR: checkoutservice:5050 Called when the user clicks 'Place Order' — triggers the full checkout orchestration.

AD_SERVICE_ADDR: adservice:9555 Fetches contextual text ads displayed at the bottom of product pages.

SHOPPING_ASSISTANT_SERVICE_ADDR: shoppingassistantservice:80 Port 80 — HTTP-based, not gRPC. All other service addresses use gRPC ports. The AI assistant uses a different protocol.

ENV_PLATFORM (commented out) Optional: specify the cloud platform (gcp, aws, azure, local, onprem, alibaba). Auto-detects if not set. Uncomment to force a specific platform.

ENABLE_PROFILER: 0 Profiler disabled. WARNING: frontend uses ENABLE_PROFILER=0 to disable, while other services use DISABLE_PROFILER=1. Opposite naming convention — easy to confuse.

CYMBAL_BRANDING, ENABLE_ASSISTANT, FRONTEND_MESSAGE, PACKAGING_SERVICE_URL Optional features available without code changes — just uncomment the env var. CYMBAL_BRANDING changes the store branding. ENABLE_ASSISTANT shows the AI chat UI. FRONTEND_MESSAGE shows a banner on all pages.

Service 1 of 2 — ClusterIP (internal)

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
```

```
  labels:
    app: frontend
spec:
  type: ClusterIP
  selector:
    app: frontend
  ports:
  - name: http
    port: 80
    targetPort: 8080
```

`name: frontend` DNS name used internally. The loadgenerator calls frontend:80 via this Service.

`type: ClusterIP` Internal only. Used by pods inside the cluster (like loadgenerator) to reach the frontend.

`name: http` Port named 'http' — correctly describes the HTTP protocol. Other services use 'grpc'.

`port: 80 / targetPort: 8080` Maps standard HTTP port 80 (external) → container port 8080 (internal). Containers on non-privileged ports (>1024) can't use port 80 directly.

Service 2 of 2 — LoadBalancer (internet-facing)

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-external
  labels:
    app: frontend
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
  - name: http
    port: 80
    targetPort: 8080
```

`name: frontend-external` A second Service for the same frontend pods. Same selector (app: frontend) — both Services route to the same pods.

`type: LoadBalancer` Tells the cloud provider (AWS/GCP/Azure) to provision an external load balancer with a PUBLIC IP address. This is how users on the internet access the store. Frontend is the ONLY service in the entire file with a LoadBalancer — everything else is ClusterIP.

`port: 80 / targetPort: 8080` Same mapping: internet traffic on port 80 → container port 8080.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: frontend
```

Frontend's Kubernetes identity. Frontend calls 8 backend services — having its own identity means you can grant it specific permissions and track its API activity separately.

11. Recommendation Service

Returns product recommendations for the 'You may also like' section. Python service that calls productcatalog to get all products, filters out current items, and returns random suggestions. Port 8080. Higher memory than Go services due to Python runtime.

Deployment

YAML

```yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: recommendationservice
  labels:
    app: recommendationservice
spec:
  selector:
    matchLabels:
      app: recommendationservice
  template:
    metadata:
      labels:
        app: recommendationservice
    spec:
      serviceAccountName: recommendationservice
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/recommendationservice:887ad4c5...
        ports:
        - containerPort: 8080
        readinessProbe:
          periodSeconds: 5
          grpc:
            port: 8080
        livenessProbe:
```

```
          periodSeconds: 5
          grpc:
            port: 8080
        env:
        - name: PORT
          value: "8080"
        - name: PRODUCT_CATALOG_SERVICE_ADDR
          value: "productcatalogservice:3550"
        - name: DISABLE_PROFILER
          value: "1"
        resources:
          requests:
            cpu: 100m
            memory: 220Mi
          limits:
            cpu: 200m
            memory: 450Mi
```

Line-by-Line Explanation

`containerPort: 8080` gRPC server on 8080. Note this is the same port as emailservice — fine because they're in separate pods.

`readinessProbe / livenessProbe.periodSeconds: 5` Both probes check every 5 seconds — faster than default 10 seconds. Recommendation should come online and stay healthy quickly.

`PRODUCT_CATALOG_SERVICE_ADDR: productcatalogservice:3550` The only downstream service recommendation calls. It fetches the full product list to find items to recommend, then filters and returns a random subset.

`requests.memory: 220Mi` Much higher than Go services (220Mi vs 64Mi). The Python CPython interpreter itself uses ~150MB of RAM just to start. This is the baseline Python overhead.

`limits.memory: 450Mi` Generous 450Mi ceiling. Recommendation loads the ENTIRE product catalog into memory for filtering. Larger catalog = more memory. The high limit prevents OOMKill under load.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: recommendationservice
  labels:
    app: recommendationservice
spec:
  type: ClusterIP
  selector:
    app: recommendationservice
  ports:
  - name: grpc
    port: 8080
```

```
    targetPort: 8080
```

Line-by-Line Explanation

`name: recommendationservice` DNS name. Frontend calls recommendationservice:8080 to get product suggestions.

`port / targetPort: 8080` Direct 1:1 mapping.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: recommendationservice
```

Isolated Kubernetes identity for recommendation pods.

12. Ad Service

Serves contextual text advertisements on product pages, matched to product category keywords. Written in Java (JVM). Port 9555. Needs significantly more startup time and resources than other services because of JVM warm-up overhead.

Deployment

YAML

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: adservice
  labels:
    app: adservice
spec:
  selector:
    matchLabels:
      app: adservice
  template:
    metadata:
      labels:
        app: adservice
    spec:
      serviceAccountName: adservice
      imagePullSecrets:
        - name: ghcr-secret
      terminationGracePeriodSeconds: 5
      securityContext:
        fsGroup: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 1000
      containers:
      - name: server
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop: [ALL]
          privileged: false
          readOnlyRootFilesystem: true
        image: ghcr.io/jaiswaladi246/adservice:b1f94434...
        ports:
        - containerPort: 9555
        env:
        - name: PORT
          value: "9555"
        resources:
          requests:
```

```
            cpu: 200m
            memory: 180Mi
          limits:
            cpu: 300m
            memory: 300Mi
        readinessProbe:
          initialDelaySeconds: 20
          periodSeconds: 15
          grpc:
            port: 9555
        livenessProbe:
          initialDelaySeconds: 20
          periodSeconds: 15
          grpc:
            port: 9555
# [END gke_release_kubernetes_manifests_microservices_demo]
```

Line-by-Line Explanation

`image SHA: b1f94434...` Unique commit SHA — adservice was deployed from its own independent git commit.

`containerPort: 9555` Custom port 9555. Avoids collision with shippingservice and paymentservice (both 50051), emailservice (8080), and recommendationservice (8080).

`No DISABLE_PROFILER env var` Adservice does NOT have this env var — unlike most other services. Either the Java profiler setup is handled differently or profiling is not configured for this service.

`requests.cpu: 200m` Higher than Go services (200m vs 100m). The JVM (Java Virtual Machine) garbage collector runs continuously in the background, consuming CPU even when the app is idle.

`requests.memory: 180Mi` Much higher than Go services (180Mi vs 64Mi). The JVM itself requires ~100-150MB just to start — before any application code runs.

`limits.cpu: 300m` Higher CPU ceiling (300m vs 200m). JVM JIT (Just-In-Time) compilation spikes CPU during the warm-up period after startup.

`limits.memory: 300Mi` 300Mi total: JVM heap + metaspace (class metadata) + thread stacks + ad service code. Java memory management is more complex than Go.

`readinessProbe.initialDelaySeconds: 20` LONGEST delay of all 12 services. Java/JVM startup involves: loading the JVM, loading application classes, running static initializers, connecting to dependencies. Go starts in milliseconds; Java takes 10-20 seconds.

`readinessProbe.periodSeconds: 15` Check every 15 seconds — SLOWEST probe frequency of all services. Java GC (garbage collection) pauses can briefly freeze the entire JVM process. A 15-second interval prevents GC pauses from being mistaken for failures.

`livenessProbe.initialDelaySeconds: 20` Same 20-second wait for liveness.

`livenessProbe.periodSeconds: 15` Same slow 15-second interval for liveness.

`# [END gke_release_kubernetes_manifests_microservices_demo]` A comment used by Google's documentation tooling to mark the end of the embeddable code snippet for their docs website. Has no effect on Kubernetes.

Service

YAML

```
---
apiVersion: v1
kind: Service
metadata:
  name: adservice
  labels:
    app: adservice
spec:
  type: ClusterIP
  selector:
    app: adservice
  ports:
  - name: grpc
    port: 9555
    targetPort: 9555
```

`name: adservice` DNS name. Frontend calls adservice:9555 to get contextual ads for product pages.

`port / targetPort: 9555` Direct 1:1 mapping.

ServiceAccount

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: adservice
```

Dedicated Kubernetes identity for adservice pods.